

Математические Методы Распознавания образов, ВМК МГУ

Optimal Control Reinforcement Learning

Автор: Афанасьев Глеб

1 Введение

Данный блок семинаров по ММРО является развитием такой области как оптимальное управление в сторону применения алгоритмов машинного обучения.

2 Задача оптимального управления и методы ее решения

§2.1 Постановка задачи оптимального управления, непрерывный и дискретный случаи

Начнем с формулировки целевой задачи данного блока: задачи оптимального управления. В общем случае, она заключается в поиске управления и порождаемой им траектории управляемого объекта, оптимальных в смысле заданного функционала и удовлетворяющих ограничениям на динамику. В общем случае, данная задача выписывается в следующем виде:

$$J(x(t), u(t)) = \int_{t_0}^{t_1} l(x(t), u(t)) dt + l_F(x(t_1)) \rightarrow \min_{x, u}$$
$$\dot{x} = f(x(t), u(t))$$

Здесь l - функция, определяющая потери, получающиеся, если в состоянии x управляемый объект выполнит действие u , l_F - функция, определяющая потери в конечном состоянии. В таком виде решением задачи является пара функций $x^*(t)$, $u^*(t)$, где $x^*(t)$ - оптимальная траектория, $u^*(t)$ - порождающее её оптимальное управление.

На практике, в силу дискретности вычислительных систем, мы чаще имеем дело с дискретным вариантом данной задачи:

$$J(x, u) = \sum_{n=1}^{N-1} l(x_n, u_n) + l_F(x_N) \rightarrow \min_{x_{1:N}, u_{1:N-1}}$$
$$x_{n+1} = f(x_n, u_n)$$

И задача таким образом сводится к поиску оптимальной траектории вида $((x_1, u_1), (x_2, u_2), \dots, (x_{N-1}, u_{N-1}), (x_N))$, минимизирующей функционал J и удовлетворяющей ограничениям на динамику. Важно помнить, что дискретной тут является только динамика, в то время как функции потерь l , l_F и f могут быть непрерывными.

Не трудно заметить, что выписанная выше задача представляет собой оптимизационную задачу с ограничениями типа "равенство".

Поиск решения этой задачи будет построен следующим образом:

1. В части 2.3 будет кратко воспроизведена теория исследования данной задачи через механизм функций Беллмана [1][2].
2. В разделе 3 будет рассмотрен практический алгоритм решения задачи в упрощенном случае, где предполагается линейный вид функции $f(x, u)$ и квадратичный вид функций $l(x, u)$ и $l_F(x, u)$. Задача оптимального управления с такими ограничениями носит название линейно-квадратичный регулятор (Linear Quadratic Regulator, LQR)
3. Затем, в разделе 4 будут сняты ограничения на вид функций динамики и потерь, и задача будет решена для произвольных функций, однако с ограничениями на пространство состояний X . В литературе рассмотренные методы носят названия Value Iteration [3] и Policy Iteration [3].
4. Наконец, в разделе 5 будет снято ограничение на пространство состояний X и рассмотрен метод, носящий в литературе название Advanced Actor Critic (A2C) [4].
5. Раздел 6 будет содержать выводы из лекций и некоторые дополнительные рассуждения

§2.2 Уравнение Беллмана

Рассмотрим наиболее популярную в наше время теорию исследования задач ОУ: теорию уравнений Беллмана. Более подробно вы её рассматривали на лекциях по оптимальному управлению [2].

2.2.1 Уравнение Беллмана

Рассмотрим общую задачу минимизации:

$$f(x) \rightarrow \min_{x \in X} \quad (2.1)$$

в теории уравнений Беллмана задача ОУ рассматривается как расширение задачи (2.1) путем добавления в нее нового параметра. Пока обозначим его как $\alpha \in \Lambda$. Теперь новая задача выглядит следующим образом:

$$F(x, \alpha) \rightarrow \min_{x \in X(\alpha)}$$

Очевидно что при каком-то $\alpha = \alpha_0$ мы приходим к оригинальной задаче:

$$F(x, \alpha_0) = f(x)$$

$$X(\alpha_0) = X$$

Введем функцию следующего вида:

$$T(\alpha) = \min_{x \in X(\alpha)} F(x, \alpha), \forall \alpha \in \Lambda$$

Функцией Беллмана называется функция T взятая со знаком минус:

$$V(\alpha) = -T(\alpha)$$

Теперь вспомним что изначально у нас стояла задача минимизации с ограничениями на динамику:

$$\begin{aligned} J(x(t), u(t)) &= \int_{t_0}^{t^1} l(x(t), u(t)) dt \rightarrow \min_{x, u} \\ \dot{x} &= f(x(t), u(t)) \end{aligned}$$

Помимо двух выписанных выше уравнений, зачастую мы хотим задать точки начала и конца траектории: $x(t_0) = x_0, x(t_1) = x_1$

Введем параметризацию данной задачи через точку начала траектории.

То есть от исходной задачи мы переходим к задаче

$$\begin{cases} J(x(t), u(t)) = \int_{t_0}^{t^1} l(x(t), u(t)) dt \\ \dot{x} = f(x(t), u(t)) \\ x(t_0) = \alpha \end{cases}$$

Конечное условие зачастую задается неявно через оптимизируемый функционал.

Соответственно, для такой задачи функция Беллмана примет следующий вид:

$$V(\alpha) = -\min_{x, u \in X_\alpha} J(x, u, \alpha)$$

Здесь и далее по конспекту следует помнить, что несмотря на то что явно мы не прописываем вхождение ограничений на динамику в функцию Беллмана, они (в отличие от советской теории ОУ с принципом максимума Понтрягина) входят туда неявно через договоренность, что траектории x, u в функции Беллмана мы рассматриваем только связанные ограничениями на динамику.

Для дальнейших рассуждений нам потребуется сделать следующие предположения о функции Беллмана и решаемой задаче:

1. Функция Беллмана существует для всех рассматриваемых α
2. $V(\alpha) \in C^1(\alpha)$
3. Любой "хвост" оптимальной траектории - оптимален

Первые два пункта обычно гарантируются видом конкретной задачи.

Рассмотрим подробнее пункт 3. По сути, он означает что если мы имеем оптимальное управление и траекторию в виде функций от времени $u^*(t), x^*(t)$ для начальной точки $x(t_0) = x_0$ и конечной точки x_1 , то это же управление будет оптимальным и для такой же задачи с начальной точкой $x_{0.5}$, где $x_{0.5}$ - точка, лежащая на оптимальной траектории $x^*(t)$.

Для рассматриваемой задачи он проверяется тривиально от противного: зафиксируем точку на траектории $x_{0.5}$ и предположим что из нее существует более оптимальное управление для достижения конечной точки x_1 . Обозначим его за $u'(t), x'(t)$ (рис 1). То есть $\int_{\tau}^{t_1} l(x'(t), u'(t))dt < \int_{\tau}^{t_1} l(x^*(t), u^*(t))dt$, где τ - момент времени когда объект оказался в точке $x_{0.5}$.

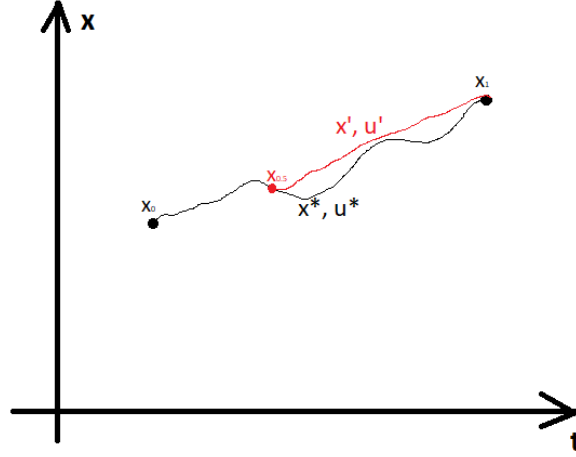


Рис. 1.

Тогда совершенно очевидно, что управление вида

$$u(t) = \begin{cases} u^*(t), t < \tau \\ u'(t), t \geq \tau \end{cases}$$

Будет более оптимальным в смысле оптимизируемого функционала, так как $\int_{t_0}^{\tau} l(x^*(t), u^*(t))dt + \int_{\tau}^{t_1} l(x'(t), u'(t))dt < \int_{t_0}^{t_1} l(x^*(t), u^*(t))dt$. Значит, управление $u^*(t)$ не оптимально, и мы приходим к противоречию с условием.

Далее будем считать, что предположения 1-3 выполнены.

Теперь рассмотрим оптимальное управление $u(t)$ для задачи выше. Оговоримся, что на курсе по оптимальному управлению было принято оперировать оптимальными парами $(x(t), u(t))$, однако, поскольку управление порождает траекторию, мы будем в основном ссылаться именно на само управление как на искомый объект, имея ввиду что он порождает соответствующую ему траекторию.

Возьмем промежуток времени Δt : $t_0 + \Delta t < t_1$.

Получаем:

$$T(\alpha) = \int_{t_0}^{t_0+\Delta t} l(x(t), u(t))dt + \int_{t_0+\Delta t}^{t_1} l(x(t), u(t))dt = \int_{t_0}^{t_0+\Delta t} l(x(t), u(t))dt + T(x(t_0+\Delta t))$$

Значит,

$$V(x(t_0 + \Delta t)) - V(\alpha) = L(t_0 + \Delta t) - L(t_0)$$

где $L(t)$ - первообразная по t для $l(x(t), u(t))$.

Делим правую и левую части на Δt и устремляем Δt к 0. Учитывая что с помощью α мы параметризовали значение $x(t_0)$ получаем:

$$\dot{V}(\alpha) = \dot{L}(t_0) = l(\alpha, u_0)$$

Вспоминаем что α может быть произвольным и должно удовлетворять заданной динамике

$$\dot{V}(\alpha) = \langle V'_x, \dot{x}(t) \rangle = \langle V'_x, f(x, u) \rangle = l(x, u)$$

$$\langle V'_x, f(x, u) \rangle - l(x, u) = 0$$

И в итоге получаем необходимое условие оптимальности:

$$\langle V'_x, f(x, u) \rangle - l(x, u) = 0$$

Теперь предположим, что мы сначала на протяжении времени Δt двигаемся каким-то случайным управлением v , а потом включаем оптимальное. Для функции T будет справедливо следующее неравенство:

$$T(\alpha) \leq \int_{t_0}^{t_0 + \Delta t} l(x(t), v(t)) dt + T(y(t_0 + \Delta t))$$

Где $y(t_0 + \Delta t)$ - точка, в которой окажется управляемый объект при включении управления v в точке α за время Δt

Переносим функцию T в левую часть и делим на Δt , получаем:

$$\frac{V(y(t_0 + \Delta t)) - V(\alpha)}{\Delta t} \leq \frac{L(t_0 + \Delta t) - L(t_0)}{\Delta t}$$

Раскладывая $y(t_0 + \Delta t)$ в ряд Тейлора, получаем:

$$y(t_0 + \Delta t) = y(t_0) + \dot{y}(t_0)\Delta t + \bar{o}(\Delta t) = \alpha + f(\alpha, v(t_0))\Delta t + \bar{o}(\Delta t)$$

И неравенство можно переписать в виде:

$$\langle V'_x, f(x, v(t)) \rangle - l(x, v) \leq 0 \quad (2.2)$$

То есть, выражение из левой части (2.2) для произвольного управления меньше 0, а для оптимального равно 0. Таким образом, мы получаем уравнение Беллмана:

$$\max_{u(t)} [\langle V'_x, f(x(t), u(t)) \rangle - l(x(t), u(t))] = 0$$

2.2.2 Переход к дискретному случаю

Так как в наше время расчеты управляющих действий и траекторий ведутся на компьютерах, где время дискретно, мы переходим к дискретной постановке задачи и рассмотрим вид изучаемой теории в дискретном случае. Напомним, что исходная задача примет вид:

$$J = \sum_{n=0}^{N-1} l(x_n, u_n) + l_F(x_N) \rightarrow \min_u$$

$$x_{n+1} = f(x_n, u_n)$$

Заметим, что далеко не всегда у нас стоит требование на фиксированное количество шагов, поэтому N часто не фиксировано и в некоторых задачах может равняться бесконечности. В таких случаях слагаемое l_F не имеет смысла.

В таких терминах, функция Беллмана $V(x)$ будет равняться минимальным по всевозможным управлениям общим потерям при условии, что мы начали движение из точки x . Переходную функцию $T(x)$ на практике пропускают.

$$V(x) = \min_u \sum_{n=0}^{+\infty} l(x_n, u_n), x_0 = x$$

Здесь суммирование идет до бесконечности, однако в случае конечных траекторий мы можем просто дополнить нулями оставшиеся награды.

Далее, вспомним как мы выводили уравнение Беллмана. Непрерывность там возникала когда мы переходили к пределу по $\Delta t \rightarrow 0$. Распишем вывод для дискретного случая когда на Δt делить не нужно:

$$V(x_k) - V(x_{k+1}) = l(x_k, u_k)$$

Так как минимально возможное количество шагов во времени равняется 1 шагу, мы получаем левую часть уравнения. Правая часть следует из того что за 1 шаг мы получим только потери за этот шаг. Вообще говоря, ровно 1 шаг делать не обязательно. Так как изначально требований на Δt не было, мы можем выписать уравнение как

$$V(x_k) - V(x_{k+n}) = \sum_{i=k}^{k+n} l(x_i, u_i)$$

2.2.3 Стохастический дискретный случай

Мы вывели дискретное уравнение Беллмана. Однако во всех выкладках мы предполагали окружающую среду строго детерминированной. В реальных задачах такое встречается редко. В большинстве случаев приходится иметь дело с некоторым элементом случайности, когда динамика принимает вид

$$x_{n+1} = f(x_n, u_n, \epsilon)$$

где ϵ - некоторый случайный параметр, взятый из распределения $p(\epsilon)$

В таком случае, функция Беллмана примет вид минимальных ожидаемых потерь с учетом случайности динамики. То есть

$$V(x) = \min_u \mathbb{E}_{x_{1.. \infty} \sim p(\epsilon)} \sum_{n=0}^{+\infty} l(x_n, u_n)$$

И уравнение Беллмана примет вид

$$\begin{aligned} V(x_k) &= \min_u \mathbb{E}_{x_{k+1.. \infty} \sim p(\epsilon)} \sum_{n=k}^{+\infty} l(x_n, u_n) = \min_u \mathbb{E}_{x_{k+1.. \infty} \sim p(\epsilon)} [l(x_k, u_k) + \sum_{n=k+1}^{+\infty} l(x_n, u_n)] = \\ &= \min_u \mathbb{E}_{x_{k+1.. \infty} \sim p(\epsilon)} [l(x_k, u_k)] + \min_u \mathbb{E}_{x_{k+1.. \infty} \sim p(\epsilon)} \left[\sum_{n=k+1}^{+\infty} l(x_n, u_n) \right] = l(x_k, u_k) + \mathbb{E}_{x_{k+1} \sim p(\epsilon)} V(x_{k+1}) \end{aligned}$$

Последнее равенство имеет место, так как мы предполагаем u_k оптимальным.

Таким образом, было выведено уравнение Беллмана в том виде, в котором его можно встретить в большинстве современной литературы [4], [5], [6], [7].

Из вида функций также можно сказать, что аналогичные соотношения имеют место для не оптимального управления. В таком случае все, что изменится - пропадут минимумы из выражений

3 Линейно Квадратичный Регулятор (LQR), уравнения Рикатти

В прошлой главе была кратко описана одна из наиболее популярных сейчас теорий исследования задач оптимального управления. Теперь рассмотрим, как эта теория может быть применена на практике.

Начнем мы с рассмотрения упрощенного случая задачи оптимального управления: линейно-квадратичного регулятора (Linear Quadratic Regulator, LQR).

С задачей LQR вы уже могли сталкиваться в курсе по оптимальному управлению. Суть его состоит в том, что мы накладываем следующие ограничения на вид динамики и функции потерь:

1. зависимость функции динамики от состояния и управления строго линейная:
 $f(x, u) = Ax + Bu$, где $A \in \mathbb{R}^{n \times n}$ и $B \in \mathbb{R}^{n \times m}$
2. зависимость функции потерь от состояния и управления строго квадратичная:
 $l(x, u) = x^T Q x + u^T R u$, где $Q \in \mathbb{R}_{++}^{n \times n}$ и $R \in \mathbb{R}_{++}^{m \times m}$

Таким образом, мы приходим к задачам следующего вида.

В непрерывном случае:

$$J(x(t), u(t)) = \int_{t_0}^{t^1} (x^T Q x + u^T R u) dt \rightarrow \min_{x, u}$$

$$\dot{x} = Ax + Bu$$

В дискретном случае:

$$J = \sum_{n=0}^{N-1} (x_n^T Q x_n + u_n^T R u_n) + x_N^T Q x_N \rightarrow \min_{x, u}$$

$$x_{n+1} = Ax_n + Bu_n$$

Для представленной системы докажем следующую теорему:

Теорема 1: Для задачи линейно-квадратичного регулятора при $N < \infty$ имеют место следующие равенства:

$$J_i^* = \min \sum_{n=i}^N (x_n^T Q x_n + u_n^T R u_n) = x_i^T P_i x_i \quad (3.1)$$

$$P_i = (A - BG_i)^T P_{i+1} (A - BG_i) + G_i^T R G_i + Q, P_N = Q \quad (3.2)$$

$$G_i = (R + B^T P_{i+1} B)^{-1} B^T P_{i+1} A \quad (3.3)$$

$$u_i^* = -G_i x_i, 0 \leq i < N \quad (3.4)$$

Доказательство: Докажем, используя математическую индукцию.

1) Для $i = N$ очевидно, что $J_N = x_N^T Q x_N$

2) Предположим, что $J_{i+1}^* = x_{i+1}^T P_{i+1} x_{i+1}$. Рассмотрим J_i^* . Согласно уравнению Беллмана $J_i^* = V(x_i) = l_i(x_i^*, u_i^*) + V(x_{i+1})$.

Тогда, учитывая определение функции Беллмана

$$J_i^* = \min_{u_i} \{x_i^T Q x_i + u_i^T R u_i + \min_{u_n} \sum_{n=i+1}^{N-1} (x_n^T Q x_n + u_n^T R u_n) + x_N^T Q x_N\} = \min_{u_i} \{x_i^T Q x_i + u_i^T R u_i + J_{i+1}^*\}$$

Учитывая предположение индукции, получаем:

$$\begin{aligned} J_i^* &= \min_{u_i} \{x_i^T Q x_i + u_i^T R u_i + x_{i+1}^T P_{i+1} x_{i+1}\} = \\ &= \min_{u_i} \{x_i^T Q x_i + u_i^T R u_i + (A x_i + B u_i)^T P_{i+1} (A x_i + B u_i)\} = \min_{u_i} h(u_i, x_i) \end{aligned}$$

Вычислим минимальное значение J_i и оптимальное u_i взяв производную:

$$\frac{\partial h(u_i, x_i)}{\partial u_i} = 2R u_i + 2B^T P_{i+1} A x_i + 2B^T P_{i+1} B u_i$$

Оптимизируемая функция выпукла, поэтому для минимальности необходимо и достаточно

$$\begin{aligned} \frac{\partial h(u_i, x_i)}{\partial u_i} &= 0 \Leftrightarrow \frac{1}{2} \frac{\partial h(u_i, x_i)}{\partial u_i} = 0 \\ R u_i + B^T P_{i+1} A x_i + B^T P_{i+1} B u_i &= (R + B^T P_{i+1} B) u_i + B^T P_{i+1} A x_i = 0 \end{aligned}$$

$$u_i^* = -(R + B^T P_{i+1} B)^{-1} B^T P_{i+1} A x_i = -G_i x_i$$

Подставим u_i^* в выражение для J_i^* , получим:

$$\begin{aligned} J_i^* &= x_i^T Q x_i + x_i^T G_i^T R G_i x_i + (A x_i + B G_i x_i)^T P_{i+1} (A x_i + B G_i x_i) = \\ &= x_i^T Q x_i + x_i^T G_i^T R G_i x_i + x_i^T (A + B G_i)^T P_{i+1} (A + B G_i) x_i = \\ &= x_i^T (Q + G_i^T R G_i + (A + B G_i)^T P_{i+1} (A + B G_i)) x_i = x_i^T P_i x_i \end{aligned}$$

теорема доказана

Уравнения (3.1)-(3.4) из теоремы называются **уравнениями Риккати [8] в дискретном случае [9]** для задачи оптимального управления с конечным горизонтом ($N < \infty$).

Перепишем уравнение (3.2) в более удобном виде:

$$\begin{aligned} P_i &= (A - B G_i)^T P_{i+1} (A - B G_i) + G_i^T R G_i + Q = \\ &= A^T P_{i+1} A - A^T P_{i+1} B G_i - G_i^T B^T P_{i+1} A + G_i^T B^T P_{i+1} B G_i + G_i^T R G_i + Q = \\ &= A^T P_{i+1} A - A^T P_{i+1} B G_i - G_i^T B^T P_{i+1} A + G_i^T (B^T P_{i+1} B + R) G_i + Q = \end{aligned}$$

учитывая (3.3)

$$= A^T P_{i+1} A - A^T P_{i+1} B G_i - G_i^T B^T P_{i+1} A + G_i^T (B^T P_{i+1} B + R) (R + B^T P_{i+1} B)^{-1} B^T P_{i+1} A + Q =$$

$$\begin{aligned}
&= A^T P_{i+1} A - A^T P_{i+1} B G_i - G_i^T B^T P_{i+1} A + G_i^T B^T P_{i+1} A + Q = \\
&= A^T P_{i+1} A - A^T P_{i+1} B G_i + Q = Q + A^T P_{i+1} A - A^T P_{i+1} B (R + B^T P_{i+1} B)^{-1} B^T P_{i+1} A = \\
&= Q + A^T (P_{i+1} - P_{i+1} B (R + B^T P_{i+1} B)^{-1} B^T P_{i+1}) A =
\end{aligned}$$

учитывая тождество Вудберри

$$P_i = Q + A^T (P_{i+1}^{-1} + B^T R^{-1} B)^{-1} A \quad (3.5)$$

Из доказанной теоремы мы очень просто можем получить точное решение задачи LQR для задачи с конечным числом шагов. Однако, возникает следующая проблема: мы не всегда наперед знаем и можем задать это самое количество шагов (гиперпараметр N). В частности, если перед нами стоит задача достижения управляемым дроном точки x_1 из точки x_0 , вычисление гиперпараметра N является весьма трудоемкой и не всегда выполнимой задачей. Проще всего будет переформулировать исходную задачу так чтобы вычисление оптимального N также легло на плечи алгоритма решения. Таким образом, мы приходим к задаче оптимального управления для бесконечного числа шагов:

$$\begin{aligned}
J &= \sum_{n=0}^{\infty} (x_n^T Q x_n + u_n^T R u_n) \rightarrow \min_{x,u} \\
x_{n+1} &= A x_n + B u_n
\end{aligned}$$

Концептуальное отличие заключается в том что в новой постановке мы предполагаем время жизни управляемого объекта бесконечным и решаем задачу, зная что функция потерь будет отрабатывать на каждом шаге до бесконечности.

Оказывается, что для новой постановки также имеют место уравнения Риккати в немного видоизмененной форме.

Теорема 2: Если задача оптимального управления с бесконечным количеством шагов имеет решение, то для него справедливы следующие соотношения:

$$u_i = -G x_i, \forall i \geq 0 \quad (3.6)$$

$$J_i^* = x_i^T P x_i, \forall i \geq 0 \quad (3.7)$$

где P - решение уравнения

$$P = Q + A^T (P^{-1} + B^T R^{-1} B)^{-1} A \quad (3.8)$$

и

$$G = (R + B^T P B)^{-1} B^T P A \quad (3.9)$$

Доказательство проходит тривиальным образом. Теорема 1 была доказана для любых натуральных N . Соответственно, если задача имеет решение, то есть существует хотя бы один набор u такой, что $\sum_{n=0}^{\infty} (x_n^T Q x_n + u_n^T R u_n) < \infty$, тогда можно сказать, что ряд $J_0^*(N)$ сходится при $N \rightarrow \infty$. Так как в теореме 1 мы получили вид $J_0^*(N) = x_0^T P_0 x_0$. Учитывая, что от значения N тут зависит только P_0 , мы получаем, что $J_0^*(N) \xrightarrow{N \rightarrow \infty} x_0^T P_{\infty} x_0$. Это значит, что P_i имеет предел при горизонте

задачи, стремящемся к бесконечности. Делая предельный переход во всех выражениях теоремы 1 и переобозначая P_∞ на P , мы получаем требуемые соотношения для x_0 .

Далее, в силу отсутствия ограничений на выбор x_0 , выведенные соотношения верны для любого x из множества состояний. **теорема доказана**

Таким образом, для решения задачи LQR теперь не обязательно делать обратный прогон, вычисляя по порядку все P_i , после чего делать прогон вперед, вычисляя u_i . Достаточно получить P и G , решив уравнения теоремы 2, и у нас будет алгоритм получения оптимального управления для любого состояния.

4 Переход к RL: алгоритмы Policy и Value Iteration

В предыдущей главе был получен алгоритм аналитического получения оптимального управления для случая когда имеются серьезные ограничения на динамику и потери, которые, вообще говоря, не всегда выполняются. Одним из выходов из данной ситуации будет линеаризация динамики, однако это не всегда реализуемо и лишает нас свободы действий при проектировании функции потерь. В этой главе мы попытаемся разобраться с этими ограничениями и найти способ решения задачи ОУ для произвольных потерь и динамики.

§4.1 Переосмысление уравнений Риккати

Начнем с обобщения механизма уравнения Риккати. Для этого рассмотрим его подробнее. В прошлой главе для случая задачи с потенциально бесконечным количеством шагов мы получили его в следующем виде:

$$P = Q + A^T(P^{-1} + B^T R^{-1} B)^{-1} A$$

Введем оператор Риккати следующим образом:

$$F(X) = Q + A^T(X^{-1} + B^T R^{-1} B)^{-1} A$$

Из двух уравнений выше следует что $P = F(P)$ Во время доказательства теоремы 2 было получено что последовательность P_0 имеет предел при $N \rightarrow \infty$. При этом, легко видеть из теоремы 1 что P_0 получается путем N -кратного применения оператора Риккати к матрице Q . Из двух обозначенных выше фактов следует

Лемма 1: Для любого $\epsilon > 0$ найдется M такое что P_{0M} , полученный не менее чем M -кратным последовательным применением оператора Риккати к матрице Q удовлетворяет следующему неравенству:

$$\|P_{0M} - P\| < \epsilon$$

На основании этой леммы мы можем сказать что решение уравнения Риккати для разрешимой задачи оптимального управления можно получить с любой заданной наперед точностью по следующему алгоритму:

Алгоритм 1

1. Инициализируем $P_{00} = Q$, $n = 0$, $\epsilon > 0$

2. Вычисляем новое значение $P_{0n+1} = F(P_{0n})$
3. Если $\|P_{0n+1} - P_{0n}\| < \epsilon$ - прекращаем, иначе $n := n + 1$ и возвращаемся к шагу 2.

Теперь рассмотрим что именно происходит на шагах 2-3 данного алгоритма.

На шаге 2 мы, основываясь на предыдущей оценке матрицы P получаем новую. При доказательстве теоремы 1 было показано что новое значение матрицы P получается как значение, минимизирующее по управлению в данной точке функцию вида

$$h(u_i, x_i) = x_i^T Q x_i + u_i^T R u_i + (Ax_i + Bu_i)^T P_{i+1} (Ax_i + Bu_i)$$

минимум которой, в свою очередь, являлся функцией Беллмана для оптимальной политики в детерминированной среде. Функция Беллмана оценивалась через значение ее же в точке куда мы придем по оптимальной политике через один шаг. Оптимальная же политика высчитывается просто как минимизатор суммы награды за ее выполнение на текущем шаге и ожидаемой награды, которую мы можем получить в состоянии куда по ней придем.

Переходя к пределу по количеству шагов мы, помимо прочего, отвызываемся от предопределенного плана на то когда и в каком состоянии мы должны оказаться, тем самым получаем универсальную формулу решения LQR в произвольной точке.

Таким образом, алгоритм 1 можно расписать следующим образом:

Алгоритм 1-Р

1. Получаем начальную аппроксимацию $P = Q$ как минимизатор функции Беллмана в тривиальном случае когда в начальный момент времени объект находится в конечном состоянии.
2. Получаем оптимальную политику действия для точки, находящейся в 1 действии от конечного состояния как минимизатор функции Беллмана в этой точке. На основании теорем 1 и 2 её можно получить по формулам (3.6) - (3.9), где P берется с предыдущего шага.
3. На основании полученной политики получаем новое значение P что, как было показано выше, эквивалентно получению значения функции Беллмана для текущей точки.
4. Получаем оптимальную политику действия для точки, находящейся в 2 действиях от целевого состояния как минимизатор функции Беллмана в этой точке. На основании теорем 1 и 2 её можно получить по формулам (3.6) - (3.9), где P берется с предыдущего шага.
5. На основании полученной политики получаем новое значение P что, как было показано выше, эквивалентно получению значения функции Беллмана для текущей точки.
6. продолжаем до сходимости

В случае с потенциально бесконечным числом шагов ($N = \infty$) у нас по сути отсутствует конечное состояние, и мы всегда предполагаем, что объект может жить вечно. Как следует из леммы 1 и теоремы 2, тогда алгоритм выше сойдется к пределу P что эквивалентно сходимости аппроксимаций функции Беллмана к значению функции Беллмана $V(x)$ для каждой точки. Политика в каждой точке вычисляется как минимизатор функции Беллмана.

Данная интерпретация алгоритма вычисления оптимального управления позволяет нам выделить в этом алгоритме те моменты, где мы пользовались линейным видом динамики и квадратичным видом функции потерь, и найти способ решить задачу без этих ограничений.

§4.2 Переход к динамике и потерям произвольного вида

Перейдем теперь к задаче оптимального управления без предположений о виде динамики и потерь. Она будет выглядеть следующим образом:

$$J(x, u) = \sum_{n=1}^{\infty} l(x_n, u_n) \rightarrow \min_{x, u}$$

$$x_{n+1} = f(x_n, u_n, \epsilon)$$

Но мы будем рассматривать ее, приняв ряд договоренностей, принятых в современном RL[3].

1. При решении задачи идет не минимизация суммарных потерь, а максимизация суммарной награды.
2. На сумму наград накладывается дисконтирование.

Таким образом, задача выглядит следующим образом:

$$J(x, u) = \sum_{n=0}^{\infty} \gamma^n l(x_n, u_n) \rightarrow \max_{x, u}, 0 < \gamma < 1$$

$$x_{n+1} = f(x_n, u_n, \epsilon)$$

В теории уравнений Беллмана для нее изменится следующее:

1. Все минимумы станут максимумами.
2. Перед оценкой функции Беллмана на следующем шаге добавится множитель γ .

В качестве решения хотелось бы найти алгоритм, аналогичный алгоритму 1, который бы позволил найти решение задачи.

Для этого обратимся к алгоритму 1-Р, расшифровывающему алгоритм 1. Предположения о виде динамики и функции потерь использовались на следующих этапах:

1. при получении начального значения P для которого доказана сходимость алгоритма

2. при явном вычислении оптимальной политики как минимизатора функции Беллмана
3. при получении нового значения P как значения, для которого будет верно уравнение Беллмана.

Как было выяснено в разделе 4.1, алгоритм 1 эквивалентен итеративному поиску значения функции Беллмана для всех состояний из X и вычислению управления как минимизатора этой функции. Специфический вид динамики и потерь тут играл роль при явном вычислении управления и функции Беллмана, а также гарантировал, что если мы имеем дело с непрерывным пространством состояний X , функция Беллмана будет всегда иметь вид $x^T P x$ при любом x для P , полученного как предел последовательности P_{0i} . Для случая произвольной динамики и потерь таких гарантий у нас нет, но они и не являлись основообразующими для алгоритма.

Попробуем переписать алгоритм 1 без предположений о динамике и потерях:

Алгоритм 2:

1. получить начальное приближение функции Беллмана $V(x)$ для всех $x \in X$
2. получить для всех x оптимальное управление $u(x)$ как максимизатор функции Беллмана $V(x) = l(x, u) + \gamma \mathbb{E}_{p(\epsilon)} V(x')$, где x' - состояние в которое мы перейдем при выполнении управления u в состоянии x .
3. получить на основании нового управления новое приближение функции Беллмана.
4. повторять до сходимости.

Для реализации этого алгоритма нам нужно получить для функции Беллмана уравнение, аналогичное уравнению (3.8) и найти итеративный способ решения. При наличии такого мы сможем реализовать пункт 1 и 3 из алгоритма 2, после чего заняться поиском управления.

Оптимальное P мы получали как решение уравнения (3.8) путем итеративного применения оператора Риккати. Попробуем найти такой же метод получения V .

Введем оператор Беллмана следующим образом: $W(V) = l(x, u) + \gamma \mathbb{E}_{p(\epsilon)} V(x')$. Имеет место следующая теорема:

Теорема 3: оператор Беллмана является сжимающим отображением для дискретного пространства состояний.

Доказательство:

$$W(V) = l(x, u) + \gamma \mathbb{E}_{p(\epsilon)} V(x') =$$

Учитывая, что пространство состояний дискретно по условию теоремы, распределение $p(\epsilon)$ также дискретно, мы можем записать:

$$W(V) = l(x, u) + \gamma \sum_{\epsilon} p(\epsilon) V(x'_\epsilon) = l(x, u) + \gamma D_\epsilon V_\epsilon$$

где D_ϵ - вектор вероятностей перейти во всевозможные состояния x , V_ϵ - вектор значений функций Беллмана для всех x , соответствующих значению ϵ . В формуле выше считается что каждое значение ϵ порождает какое-то следующее состояние x' .

$$||W(V_1) - W(V_2)|| = ||l(x, u) + \gamma D_\epsilon V_1 - l(x, u) - \gamma D_\epsilon V_2|| = ||\gamma D_\epsilon V_1 - \gamma D_\epsilon V_2|| \leq \gamma ||D_\epsilon|| * ||V_1 - V_2||$$

Чтобы $W(V)$ было сжимающим отображением, нужно показать что $||D_\epsilon|| \leq 1$. Воспользуемся тем фактом, что на данный момент мы норму используем только в доказываемой теореме и в определении сходимости P , где ограничений на норму не накладывалось. Договоримся о виде нормы.

$$||D_\epsilon|| = ||D_\epsilon||_\infty = \max_i |D_{\epsilon i}|$$

Так как P_ϵ - это вектор вероятностей, $||P_\epsilon||_\infty \leq 1$

Тогда

$$||W(V_1) - W(V_2)|| \leq \gamma ||P_\epsilon|| * ||V_1 - V_2||$$

где $\gamma ||P_\epsilon|| < 1$, следовательно оператор Беллмана - сжимающее отображение.

Теорема доказана.

Теорема 3 значит что если мы будем применяя оператор Беллмана последовательно обновлять значения функции Беллмана во всех состояниях и брать политику как ее максимизатор, в какой-то момент мы сойдемся к оптимальному решению.

Таким образом, результатом данного раздела является алгоритм Iterative Policy Evaluation:

1. Инициализируем δ . Для всех состояний x инициализируем $V(x)$,
2. Для всех состояний x и оптимальной политики u : $v := V(x)$, $V(x) := r(x, u) + \gamma \mathbb{E}_{p(\epsilon)} V(x')$, $\Delta_x := |v - V(x)|$
3. Повторяем шаг 2 хотя бы для одного x $\Delta_x > \delta$

§4.3 Policy и Value Iteration

Итак, мы провели все подготовительные работы и готовы сформулировать первые два метода из области обучения с подкреплением.

Policy Iteration:

1. Инициализируем $V(x)$, $u(x)$.
2. С помощью Iterative Policy Evaluation обновляем значения $V(x)$.
3. Для всех x обновляем $u(x) = \operatorname{argmax}_u \{r(x, u) + \gamma \mathbb{E}_{p(\epsilon)} V(x')\}$
4. Если на предыдущем шаге хотя бы одно управление изменилось, то возвращаемся к шагу 2.

Учитывая вид управления, мы можем объединить шаги 2 и 3 и получить еще один алгоритм.

Value Iteration:

1. Инициализируем $V(x)$, $u(x)$.

2. Обновляем значения $V(x)$ по формуле: $V(x) = \max_u \{r(x, u) + \gamma \mathbb{E}_{p(\epsilon)} V(x')\}$
3. Если на предыдущем шаге хотя бы одно управление изменилось, то возвращаемся к шагу 2.

Разумеется, в этих алгоритмах мы делаем сильное предположение о виде пространства состояний X : оно должно быть дискретным. Как снять это ограничение, будет рассказано в следующей главе.

5 Advanced Actor Critic

В прошлых главах были выполнены все подготовительные работы, и теперь мы можем перейти к выводу одного из популярных в наше время методов обучения с подкреплением: Advanced Actor Critic (A2C).

Последние два метода RL, которые были выведены - это Policy и Value Iteration. Суть их заключается в том что они итеративно применяя оператор Беллмана, вычисляют функцию Беллмана для всех состояний и управлением считается действие, переводящее агента в состояние где значение функции Беллмана максимально. Серьезным ограничением данных методов является необходимость дискретности пространства состояний (X). Очевидно, это не всегда так. Разумеется, пространство можно дискретизовать, но тогда может потеряться много информации о нем. Рассмотрим другой способ.

§5.1 RL-ные обозначения

Для начала перейдем от терминов и обозначений, принятых в ОУ к таковым в обучении с подкреплением. Мы уже частично начали это в прошлой главе. Закончим процесс.

1. В RL принято не минимизировать потери, а максимизировать награду. Функцию, определяющую награду за то что в состоянии x_i применил управление u_i принято обозначать как $r(x_i, u_i)$.
2. В RL принято состояние обозначать буквой s (сокращение от state), а действие - буквой a (сокращенно от action).
3. В RL принято действие объекта предсказывать не в виде действия, а в виде вероятностного распределения по всему пространству действий A , откуда семплируется действие на каждом шаге при условии текущего состояния.
4. (следствие из пункта 3) В RL принято разделять параметр ϵ , отвечающий за непостоянство поведения управляемого объекта на случайность динамики среды и случайность управления,
5. В RL принято управление называть политикой.

Таким образом, мы переходим к задаче следующего вида:

$$J(s, a) = \sum_{n=0}^{\infty} \gamma^n r(s_n, a_n) \rightarrow \max_{s,a}, 0 < \gamma < 1$$

$$a \sim \pi(a|s), s_{n+1} \sim p(s|s_n, a_n)$$

Отличия эти, на самом деле, исключительно косметические, однако для удобства ориентации читателя в литературе по RL, вывод следующих методов будет проходить в новых обозначениях.

§5.2 Метод REINFORCE

Выпишем функцию Беллмана для новой постановки:

$$V(s) = \max_{\pi} \mathbb{E}_{a \sim \pi(a|s), s_{n+1} \sim p(s|s_n, a_n)} \sum_{n=0}^{\infty} \gamma^n r(s_n, a_n), s_0 = s$$

Задача RL теперь состоит в поиске такого распределения $\pi(a|s)$, при котором суммарная награда по пройденной траектории максимальна. В предыдущей части мы вычисляли явно функцию Беллмана для всех состояний и выводили политику как траекторию, проходящую по состояниям с максимальным значениям этой функции. Однако, сейчас и пространство состояний S и пространство действий A могут быть непрерывными.

Одним из решений кажется ввести параметризацию распределения $\pi(a|s) = \pi(a|s, \theta)$ и подобрать (или обучить) параметры таким образом, чтобы семплируя действия из этого распределения, мы получали всегда оптимальное действие. Но тогда возникают два вопроса:

1. Откуда взять $V(s)$ в непрерывном случае?
2. Как учить $\pi(a|s, \theta)$?

Второй вопрос возникает исходя из того, что даже если у нас будет V , для обучения π потребуется:

1. Получить действия $a \sim \pi(a|s, \theta)$,
2. Получить функцию $\bar{V}(a) = \sum_{s' \sim p(s'|s, a), a \sim \pi(a|s, \theta)} r(s, a)$,
3. Вычислить $\frac{\partial \bar{V}}{\partial a}$ и пропустить градиент $\frac{\partial a}{\partial \theta}$ по всем параметрам обучаемой функции чтобы учить ее на максимизацию V .

Однако, на практике далеко не всегда мы знаем динамику и можем выписать ее в явном виде. Например, если s - это вектор высокой размерности, составленный из показаний визуальных сенсоров управляемого объекта, а a - вектор напряжений, которые подаются в каждый момент на моторы объекта. Установление связи между ними в виде математической функции или вероятностного распределения - чрезвычайно сложная задача. Кроме того, требуется как-то провести градиент через математическое ожидание.

Для того чтобы понять, как решить эти проблемы, введем вспомогательные функции:

$$V_{\pi}(s) = \mathbb{E}_{a \sim \pi(a|s), s_{n+1} \sim p(s|s_n, a_n)} \sum_{n=0}^{\infty} \gamma^n r(s_n, a_n), s_0 = s$$

$$Q_\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(s'|s, a)} V_\pi(s')$$

Функция $V_\pi(s)$ от функции Беллмана отличается лишь тем, что тут предполагается получение действий не из оптимальной политики, а из текущей рассматриваемой. То есть эта функция показывает ожидаемую награду, если в состоянии s действия будут семплироваться конкретно из распределения π . В главе 2 было упомянуто, что для такой функции также имеет место соотношение, похожее на уравнение Беллмана. Введение такой функции позволит нам оценить то, насколько хороша наша политика и как близко мы подошли к функции Беллмана.

Q_π -функция - функция, показывающая ожидаемую итоговую награду, если в состоянии s будет применено действие a , после чего включится политика π . Ее введение позволит нам оценить то, как изменится качество политики при изменении действия на данном шаге.

Для определенности далее будем предполагать, что $\pi(a|s, \theta)$ - некое распределение, параметры которого предсказываются нейросетью с весами θ .

Для обучения параметров на максимизацию ожидаемой награды, которая будет получена при генерации управления из π , нам потребуется вычислить градиент $\nabla_\theta V_\pi(s)$.

Для начала, выразим Q_π через V_π и раскроем математическое ожидание:

$$\nabla_\theta V_\pi(s) = \nabla_\theta \mathbb{E}_{a \sim \pi(a|s, \theta)} Q_\pi(s, a) = \nabla_\theta \int_A \pi(a|s, \theta) Q_\pi(s, a) da =$$

Вносим градиент под интеграл:

$$\begin{aligned} &= \int_A \nabla_\theta \pi(a|s, \theta) Q_\pi(s, a) da + \int_A \pi(a|s, \theta) \nabla_\theta Q_\pi(s, a) da = \\ &= \int_A \frac{\nabla_\theta \pi(a|s, \theta)}{\pi(a|s, \theta)} \pi(a|s, \theta) Q_\pi(s, a) da + \int_A \pi(a|s, \theta) \nabla_\theta Q_\pi(s, a) da = \\ &= \int_A \nabla_\theta \log(\pi(a|s, \theta)) \pi(a|s, \theta) Q_\pi(s, a) da + \int_A \pi(a|s, \theta) \nabla_\theta Q_\pi(s, a) da = \\ &= \mathbb{E}_a[\nabla_\theta \log(\pi(a|s, \theta)) Q_\pi(s, a) + \nabla_\theta Q_\pi(s, a)] = \\ &= \mathbb{E}_a[\nabla_\theta \log(\pi(a|s, \theta)) Q_\pi(s, a) + \nabla_\theta (r(s, a) + \gamma \mathbb{E}_{s'} V_\pi(s'))] = \end{aligned}$$

так как $r(s, a)$ не зависит от θ

$$= \mathbb{E}_a[\nabla_\theta \log(\pi(a|s, \theta)) Q_\pi(s, a) + \gamma \nabla_\theta (\mathbb{E}_{s'} V_\pi(s'))] =$$

выносим матожидание по состояниям

$$= \mathbb{E}_{a, s'}[\nabla_\theta \log(\pi(a|s, \theta)) Q_\pi(s, a) + \gamma \nabla_\theta V_\pi(s')]$$

В итоге получаем следующую формулу:

$$\nabla_\theta V_\pi(s) = \mathbb{E}_{a, s'}[\nabla_\theta \log(\pi(a|s, \theta)) Q_\pi(s, a) + \gamma \nabla_\theta V_\pi(s')]$$

Таким образом, была выражена зависимость $\nabla_\theta V_\pi(s)$ от $\nabla_\theta V_\pi(s')$, то есть от градиента этой же функции но для следующего состояния. Очевидно что $\nabla_\theta V_\pi(s')$

мы можем расписать аналогичным образом и получить ее выражение уже для следующего после s' состояния. Продолжать мы это можем до бесконечности. Запишем что получится в итоге:

$$\nabla_{\theta} V_{\pi}(s) = \mathbb{E}_{\tau} \sum_{n=0}^{\infty} [\gamma^n \nabla_{\theta} \log(\pi(a_n | s_n, \theta)) Q_{\pi}(s_n, a_n)]$$

где τ - траектория, полученная при семплировании действий из π .

Теперь градиент функции $V_{\pi}(s)$ принял более вычисляемый вид, и вообще говоря, на этом этапе можно остановиться и заняться поиском приближенных значений Q_{π} и методов аппроксимации выведенного выражения. Однако, это крайне неудобно делать вручную и нам бы хотелось чтобы у нас была лосс-функция, градиент от которой был бы равен вычисленному выше выражению. Тогда мы бы смогли посчитать ее в pytorch и вызвать backward. Очевидно что функция вида $\mathbb{E}_{\tau} \sum_{n=0}^{\infty} [\gamma^n \log(\pi(a_n | s_n, \theta)) Q_{\pi}(s_n, a_n)]$ на эту роль не подойдет, так как если от нее мы возьмем градиент по θ , мы не получим требуемого градиента, ведь θ в нее входит не только через логарифм, но и через математическое ожидание и Q_{π} . Попробуем найти лосс в другом виде. Для этого рассмотрим функцию вида

$$L^{\bar{\pi}}(\pi) = \mathbb{E}_{\tau \sim \bar{\pi}} \sum_{n=0}^{\infty} [\gamma^n \log(\pi(a_n | s_n, \theta)) Q_{\bar{\pi}}(s_n, a_n)]$$

Она обладает следующими свойствами:

1. $L^{\bar{\pi}}(\pi) = V_{\pi}(s)$ при $\bar{\pi} = \pi$,
2. $\nabla_{\theta} L^{\bar{\pi}}(\pi) = \nabla_{\theta} V_{\pi}(s)$ при $\bar{\pi} = \pi$

Это значит что если мы зафиксируем текущее π , оценим по нему математическое ожидание и Q , после чего градиент $\nabla_{\theta} V_{\pi}(s)$ будем брать считая что от θ зависит только $\log(\pi(a_n | s_n, \theta))$, мы будем получать градиент такой же как если бы честно дифференцировали всю $V_{\pi}(s)$.

Таким образом, мы готовы выписать новый RL-алгоритм под названием REINFORCE:

1. Инициализируем θ
2. Заставляем агента в среде действовать, семплируя управление из $\pi(a | s, \theta)$, собираем M траекторий: $\tau_0, \tau_1, \tau_2, \dots, \tau_M$. Тут стоит отметить что хотя при выводе формул мы предполагали симуляцию бесконечной, чаще перед нами стоит задача по достижению какого-то состояния и удержания его на протяжении какого-то времени. Поэтому, даже в задачах с условно бесконечной траекторией, траектории симулируются конечные, просто очень длинные. Считается что если объект достиг и удержал целевое состояние на протяжении времени, то он справится с задачей и на бесконечно большом промежутке времени.
3. Вычисляем оценки $\bar{Q}_{\pi}(s_n, a_n) = \sum_{k \geq n} \gamma^{k-n} r(s_k, a_k)$
4. Вычисляем оценку $\overline{\nabla_{\theta} V_{\pi}(s)} = \frac{1}{M} \sum_{\tau_i} \sum_{n=0}^{|\tau_i|} \gamma^n \nabla_{\theta} \log(\pi(a_n | s_n, \theta)) \bar{Q}_{\pi}(s_n, a_n)$

5. Улучшаем параметры: $\theta \leftarrow \theta + \alpha \overline{\nabla_{\theta} V_{\pi}(s)}$
6. Повторяем шаги 2-5 до сходимости

Очевидно, что серьезным недостатком данного алгоритма является необходимость собирания полных траекторий $\tau_0, \tau_1, \tau_2, \dots, \tau_M$. Это займет большое количество времени. При этом, после выполнения же шага 5, политика поменяется, и нам нужно будет снова собрать траектории, так как старые уже не релевантны. Кроме того, у оценки $\overline{\nabla_{\theta} V_{\pi}(s)}$ чрезвычайно большая дисперсия [10], что еще сильнее замедлит обучение. Для того чтобы решить эти проблемы, улучшим метод REINFORCE.

§5.3 A2C

Итак, в предыдущей части мы смогли вывести вычисляемый лосс, максимизация которого эквивалентна максимизации ожидаемой награды. Иными словами, мы смогли решить вторую из двух проблем, обозначенных в начале части 5.2. Однако, при ее решении мы использовали лишь суммарные награды по траектории и никак не затрагивали свойства функции Беллмана. Рассмотрим подробнее, что мы имеем на данный момент:

1. Задача поиска оптимального управления была сведена к задаче оптимизации нейросети за счет вероятностной постановки и лосс-функции специфического вида,
2. Лосс-функция вычисляется в непрерывном пространстве состояний и действий,
3. Вычисление лосс-функции основано на Монте-Карло оценке функции Q_{π} , что по сути является Монте-Карло оценкой функции V_{π}

У данного подхода есть ряд недостатков. В данной главе мы поработаем с двумя из них, а именно: с большой дисперсией оценки $\overline{\nabla_{\theta} V_{\pi}(s)}$ и с необходимостью управлять объектом до момента завершения его задачи для сбора траекторий для оценки. Начнем по порядку.

Выпишем еще раз функцию

$$\nabla_{\theta} V_{\pi}(s) = \mathbb{E}_{\tau} \sum_{n=0}^{\infty} [\gamma^n \nabla_{\theta} \log(\pi(a_n | s_n, \theta)) Q_{\pi}(s_n, a_n)]$$

Посмотрим на математическое ожидание $\mathbb{E}_a \nabla_{\theta} \log(\pi(a | s, \theta))$. Мы можем записать что:

$$\begin{aligned} \mathbb{E}_a \nabla_{\theta} \log(\pi(a | s, \theta)) &= \int_A \nabla_{\theta} \log(\pi(a | s, \theta)) \pi(a | s, \theta) da = \\ &= \int_A \frac{\nabla_{\theta} \pi(a | s, \theta)}{\pi(a | s, \theta)} \pi(a | s, \theta) da = \int_A \nabla_{\theta} \pi(a | s, \theta) da = \nabla_{\theta} \int_A \pi(a | s, \theta) da = \nabla_{\theta} 1 = 0 \end{aligned}$$

То есть, градиент $\pi(a | s, \theta)$ меняется в окрестностях 0. В то же время, Q_{π} по определению является суммой наград, которые в свою очередь ограничены лишь

фантазией инженера и могут быть очень большими, особенно в случае бесконечных или очень длинных траекторий. В связи с этим, дисперсия Монте-Карло оценки такой величины очень большая [10].

Решением данной проблемы будет нормировать Q_π , вычтя из нее какую-либо не зависящую от θ функцию.

$$\nabla_\theta V_\pi(s) = \mathbb{E}_\tau \sum_{n=0}^{\infty} [\gamma^n \nabla_\theta \log(\pi(a_n|s_n, \theta)) [Q_\pi(s_n, a_n) - b(s_n)]]$$

Аналитически оптимальная $b(s)$ выведена в [10] и равняется $\frac{\mathbb{E}_a[||\nabla_\theta \log(\pi(a|s, \theta))||^2 Q_\pi(s, a)]}{\mathbb{E}_a[||\nabla_\theta \log(\pi(a|s, \theta))||^2]}$

На практике вычислять данное выражение сложно и часто используют костыль: предполагают, что $\nabla_\theta \log(\pi(a|s, \theta))$ на всем пространстве состояний примерно похожи друг на друга [10], [4], в силу чего можно записать:

$$\frac{\mathbb{E}_a[||\nabla_\theta \log(\pi(a|s, \theta))||^2 Q_\pi(s, a)]}{\mathbb{E}_a[||\nabla_\theta \log(\pi(a|s, \theta))||^2]} \approx \mathbb{E}_a Q_\pi(s, a) = \bar{V}_\pi(s)$$

Таким образом, можно записать что:

$$\nabla_\theta V_\pi(s) = \mathbb{E}_\tau \sum_{n=0}^{\infty} [\gamma^n \nabla_\theta \log(\pi(a_n|s_n, \theta)) [Q_\pi(s_n, a_n) - \bar{V}_\pi(s_n)]]$$

Разность $[Q_\pi(s_n, a_n) - \bar{V}_\pi(s_n)]$ принято обозначать символом $A_\pi(a_n, s_n)$ как сокращение от Advantage. На практике получается что данная модификация серьезно уменьшает дисперсию [10], [4].

Теперь разберемся с проблемой оценки $Q(s, a)$. По определению, $Q_\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(s'|s, a)} V_\pi(s')$. Учитывая что V_π представляет собой сумму наград, новую оценку

$$\bar{\bar{Q}}_\pi(s, a) = r(s, a) + \gamma r(s', a') + \gamma^2 r(s'', a'') + \dots + \gamma^N V_\pi(s^{(N)})$$

И

$$\bar{\bar{A}}_\pi(s, a) = r(s, a) + \gamma r(s', a') + \gamma^2 r(s'', a'') + \dots + \gamma^N V_\pi(s^{(N)}) - V_\pi(s)$$

И теперь мы снова возвращаемся к вопросу из начала данной главы. Откуда взять V ? В части 4.2 мы использовали уравнение Беллмана для аналитического вычисления функций Беллмана во всех состояниях и получения из них оптимального управления. Здесь мы не можем проитерироваться по всем состояниям в силу непрерывности пространства. Возникает идея: если нельзя проитерироваться, можно попробовать аппроксимировать эту функцию нейросетью и обучить ее по стандартной схеме машинного обучения на ее значениях в точках. Аппроксимировать V_π будем нейросетью как $V_\pi(s|\varphi)$. Обучать параметры φ будем так, чтобы $V_\pi(s|\varphi)$ удовлетворяла уравнению Беллмана, которое является еще и необходимым условием оптимальности порождающего управления. Таким образом, мы приходим к новому алгоритму:

Алгоритм A2C:

1. Инициализируем 2 нейронные сети $\pi(a|s, \theta)$ и $V(s|\varphi)$

2. Получаем из динамики несколько кусков траектории, реализуя π .
3. Вычисляем лосс для нейросети π : $L_\pi = -\frac{1}{MN} \sum_{m=0}^M \sum_{t=0}^N \log((a_t^m | s_t^m)) A_\pi(a_t^m, s_t^m)$,
4. Вычисляем таргет для нейросети во всех состояниях траектории $V(s|\varphi)$: $y(s_t^m, a_t^m) = \sum_{t'=t}^{N-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) + \gamma^{N-t} V_\pi(s_N)$, где $(N-1) \geq t$,
5. Вычисляем лосс для нейросети $V(s|\varphi)$: $L_V = \frac{1}{MN} \sum_{m=0}^M \sum_{t=0}^N (V(s_t^m) - y(s_t^m, a_t^m))$,
6. Делаем градиентный шаг для обеих сетей,
7. Продолжаем до сходимости.

Выписанный выше алгоритм носит название Advantage Actor Critic[4]. В данный момент он вместе со своими модификациями является одним из наиболее популярных методов обучения с подкреплением.

6 Итог

Несколько слов в заключение. На протяжении данного конспекта была разобрана теория оптимального управления, рассмотрены методы её решения и было показано, как эта задача переформулируется в термины RL и решается через нейросети. Несмотря на то, что классические методы ОУ кажутся устаревшими, в современном мире они имеют широкое применение, в частности в [11], [12], [13], [14]. Происходит это потому, что существующие на сегодняшний день технологии RL не позволяют аккуратно балансировать между решением задачи, возможностью реализации алгоритма, обученного в симуляторе, на реальном железе и универсальностью обученной нейросети. Так как разработка наград всегда лежит на совести конкретного инженера, а сеть при обучении заточивается именно на их максимизацию, можно оказаться в ситуации, когда максимизация наград не всегда соответствует реальному выполнению поставленной задачи. Кроме того, зачастую возникает проблема, что простым вводом штрафа за смерть мы не обучим агента избегать опасных ситуаций, так как они могут возникать в результате самых разных сценариев, не связанных друг с другом (перегрев моторов, поломка двигателя, большое напряжение на моторах, создающее большую разность потенциалов и зашумляющее сенсоры (к вопросу о том, зачем ВМКашнику электрод), разное поведение при разном количестве топлива и т.д.), и простым штрафом за факт смерти тут не обойтись. Все это приводит к тому, что с помощью RL получается достичь устойчивого поведения объекта, но его весьма трудно адаптировать под иные условия и задачи.

У оптимального управления другая проблема. Решение задач с помощью него приводит к достижению требуемого результата с высокой точностью и возможностью адаптации под другие задачи, но оно требует громоздких выкладок и часто моделируется в оффлайн-режиме, что приводит к большой неустойчивости при малейшем отклонении от планируемого сценария.

И все это накладывается на то, что симуляция динамики всегда отличается от реальности и возникает так называемый sim2real gap [15].

Поэтому, при решении задач управления необходимо учитывать перечисленные плюсы и минусы и находить правильный баланс. Работа RL-щика сейчас в основном

закljučается именно в изучении физического устройства кинематики и динамики управляемого объекта и поиске способов переформулировать это все в систему наград и штрафов и их правильную настройку.

Список литературы

- [1] [Richard Bellman. Dynamic programming](#)
- [2] [Лекция Орлова про уравнение Беллмана](#)
- [3] [Richard S. Sutton and Andrew G. Barto, Reinforcement Learning, глава 4](#)
- [4] [Asynchronous Methods for Deep Reinforcement Learning](#)
- [5] [Trust Region Policy Optimization](#)
- [6] [Proximal Policy Optimization Algorithms](#)
- [7] [Soft Actor-Critic](#)
- [8] [Riccati Equations in Optimal Control Theory](#)
- [9] [New Forms of Riccati Equations and the Further Results of the Optimal Control for Linear Discrete-Time Systems](#)
- [10] [Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning](#)
- [11] [SLIP Nature Embodied Robust Quadruped Robot Control](#)
- [12] [Tello Leg: The Study of Design Principles and Metrics for Dynamic Humanoid Robots](#)
- [13] [Prioritized Motion-Force Control of Constrained Fully-Actuated Robots: Task Space Inverse Dynamic](#)
- [14] [Online Trajectory Optimization for Dynamic Aerial Motions of a Quadruped Robot](#)
- [15] [Пример про sim2real](#)
- [16] [Лекция про A2C](#)
- [17] [Статья про связь VI и PI с уравнения Риккати](#)